PyVista review: the arrays behind the pictures

Every mesh in this folder is arrays of numbers. PyVista draws those arrays. This guide keeps the real names and the real array shapes, and says what each number means.

An array

A NumPy array is a block of numbers with a shape. Shape (100, 3) means 100 rows and 3 columns. Shape (100,) means 100 numbers in one line. You index with square brackets: points[0] is the first row, points[:, 2] is the whole z column, points[:5, :] is the first 5 rows.

The points array: shape (n_points, 3)

Each row is one point in 3D. Column 0 is x, column 1 is y, column 2 is z. The row number is the point's index. Later arrays refer to points by that index, not by copying the coordinates.

```
points = np.random.rand(100, 3)   # 100 points, random x,y,z in [0, 1)
points[:5, :]                       # look at the first 5 rows
mesh = pv.PolyData(points)          # a mesh that is only those points
```

points array, shape (4, 3) — one row is one point

	x	y	z
0	0.0	0.0	0.0
1	1.0	0.0	0.0
2	1.0	1.0	0.2
3	0.0	1.0	0.0

faces array for one triangle

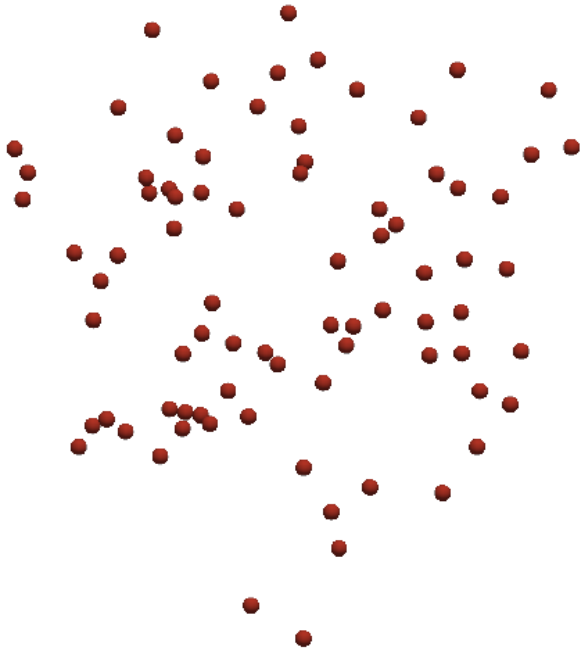
```
[ 3,  0,  1,  2 ]
```

3 = this cell has 3 points

0, 1, 2 = row numbers in points

A second triangle would be appended: [3, 0, 2, 3]

Left: four points. The red number is the row index. Right: a faces array that says "one triangle, using rows 0, 1, and 2."



Those rows drawn with style="points". No faces array yet, so nothing connects the dots.

The faces array: indices into points

A face says which rows of points form one cell. The format is a flat 1D array. For each cell you write the count, then that many indices.

```
faces = np.array([3, 0, 1, 2])     # one triangle: 3 corners, rows 0, 1, 2
# two triangles would be concatenated:
faces = np.array([3, 0, 1, 2,  3, 0, 2, 3])
surf = pv.PolyData(points, faces)
```

A line cell starts with 2. A quad starts with 4. The number at the start is how many indices follow, not a point index. In the sphere widget notebooks the surface is built exactly this way: surf.points is a (n, 3) array and surf.faces is np.array([3, 0, 1, 2]).

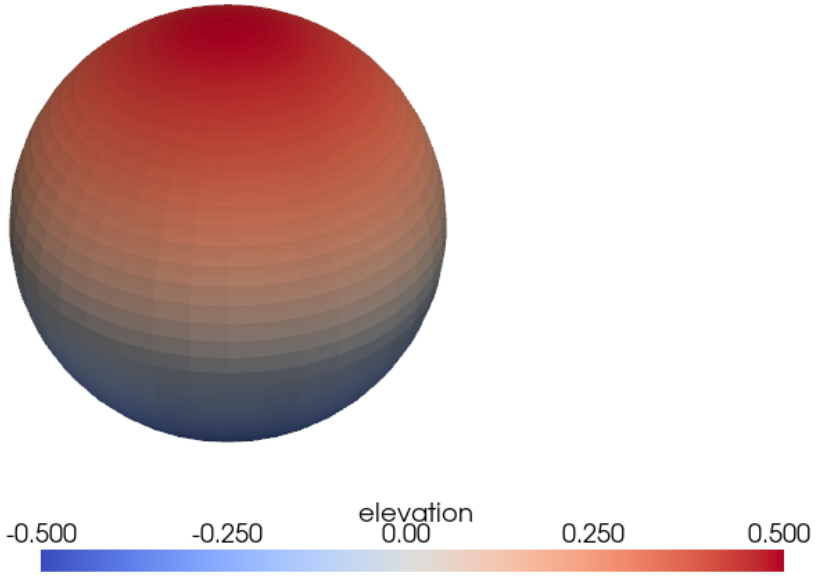
02_mesh: point data, cell data, field data

Once the geometry exists, you attach more arrays. The length of the array has to match what it describes.

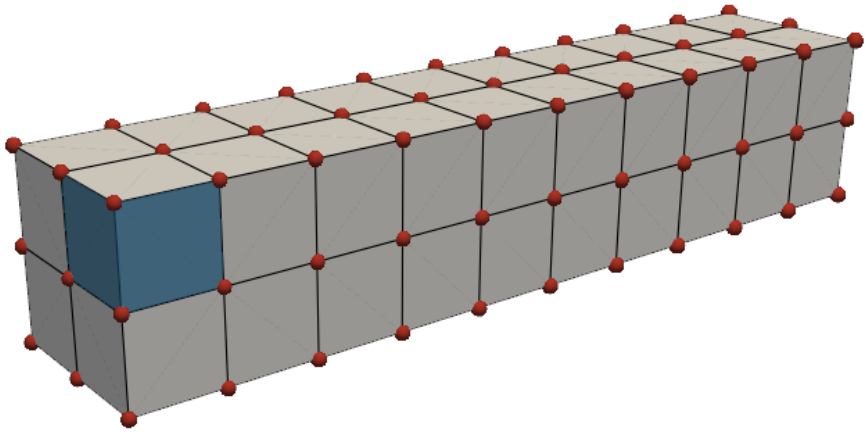
- Point data: one value per point. Length must equal mesh.n_points. Stored on mesh.point_data.
- Cell data: one value per cell. Length must equal mesh.n_cells. Stored on mesh.cell_data.
- Field data: one array for the whole mesh. Its length is not tied to points or cells. mesh.field_data.

```
mesh.point_data["my point values"] = np.arange(mesh.n_points)
mesh.cell_data["my cell values"] = np.arange(mesh.n_cells)
mesh.plot(scalars="my point values")
```

np.arange(n) is the array [0, 1, 2, ..., n-1]. scalars="name" tells the plotter which of those arrays to turn into color. A cube has 8 points and 6 faces, so the lesson assigns range(8) as point data and range(6) as cell data. Same mesh, different array length, different coloring.



Point data of shape (n_points,). Here the array is the z column of the points, so color follows height.



mesh.points is still an (n_points, 3) array (red). Cells connect those rows. One cell is highlighted.

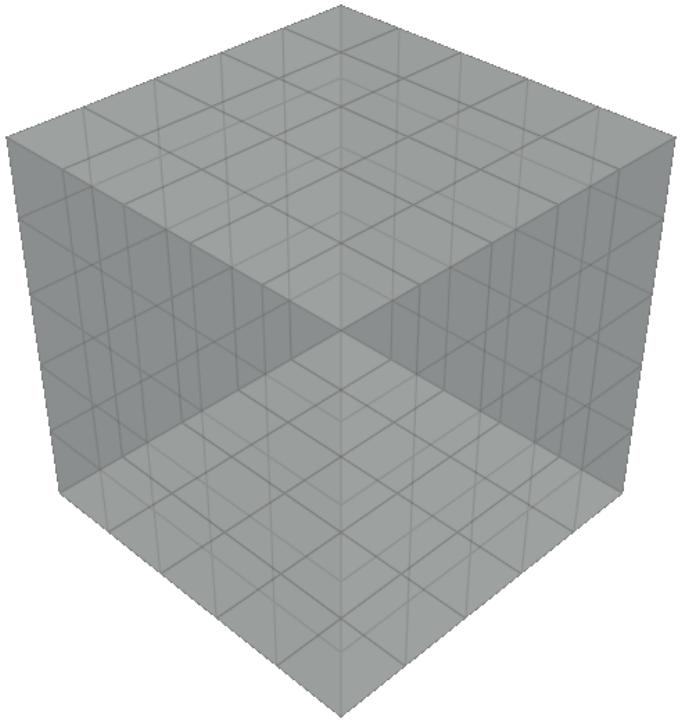
A 3D array becomes a grid: ImageData

A volume is often a 3D NumPy array, shape (nx, ny, nz). ImageData does not store a points row for every sample. It stores three numbers that generate the points:

- dimensions: how many points along x, y, z. For point data this equals the array shape. For cell data it is the array shape plus 1, because cells sit between points.
- spacing: distance between points along each axis, a length-3 array.
- origin: the x,y,z of the first point, a length-3 array.

```
values = np.linspace(0, 10, 1000).reshape((20, 5, 10))  # shape (20, 5, 10)
grid = pv.ImageData()
# cell data: one value per cell, so one more point than cells on each axis
grid.dimensions = np.array(values.shape) + 1               # (21, 6, 11)
grid.cell_data["values"] = values.flatten(order="F")
# point data: dimensions match the array, and you flatten the other way around
grid.dimensions = values.shape                             # (20, 5, 10)
grid.point_data["values"] = values.flatten(order="F")
```

flatten(order="F") walks the array in Fortran order, which is the order VTK expects (x changes fastest). If you forget order="F", the numbers land on the wrong points and the picture looks scrambled even though the values are right. ravel(order="F") is the same idea.



A small ImageData. dimensions sets how many points; spacing sets the gap. The picture is those two arrays, not a hand-written (n, 3) points list.

Building a surface from three 1D arrays

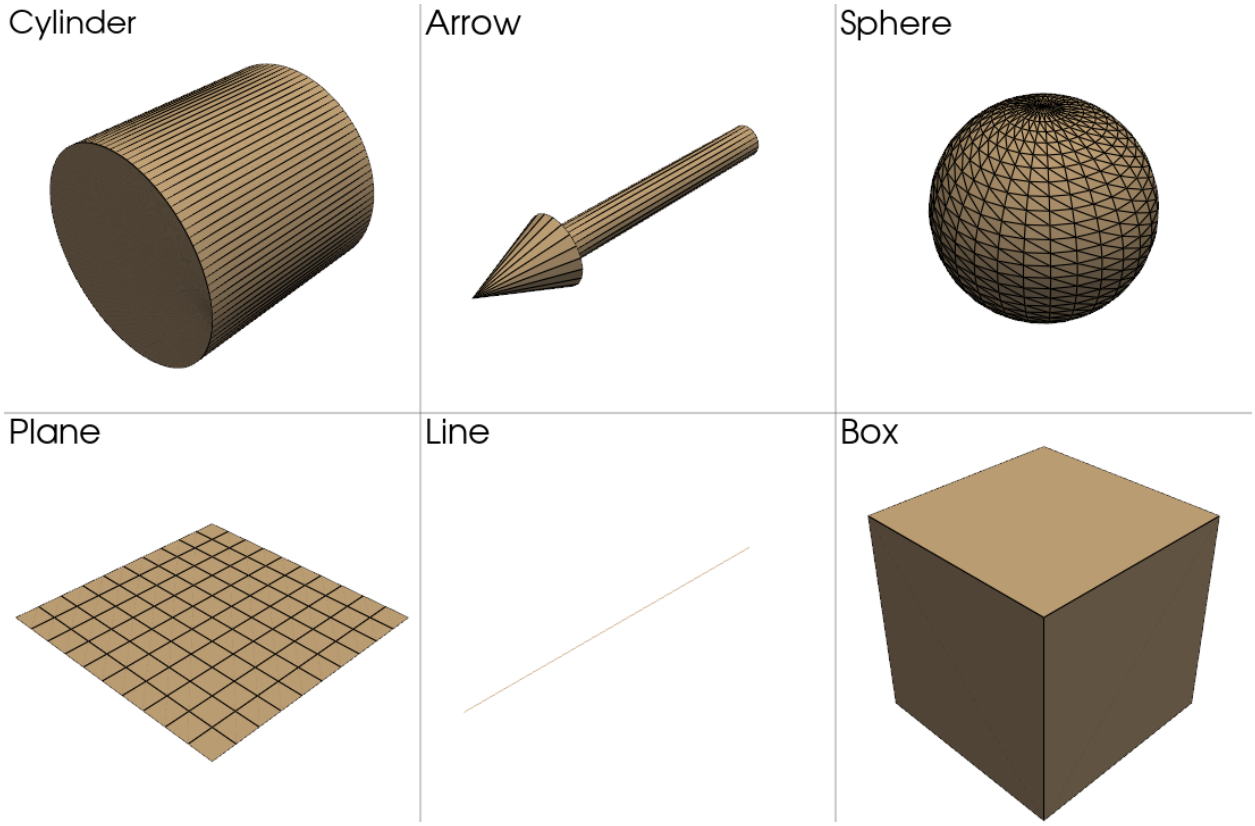
The triangulated-surface exercise starts with coordinates, not a file.

```
x = np.linspace(-200, 200, num=20)     # shape (20,)
y = np.linspace(-200, 200, num=20)     # shape (20,)
xx, yy = np.meshgrid(x, y)             # each becomes shape (20, 20)
zz = 100 * np.exp(-0.5 * ((xx/100)**2 + (yy/100)**2))
points = np.c_[xx.reshape(-1), yy.reshape(-1), zz.reshape(-1)]
# points.shape is (400, 3)
```

meshgrid pairs every x with every y. np.c_ stacks the flattened columns side by side into the (n, 3) points array. You can then call a filter such as delaunay_2d, which builds the faces array for you from those points.

00 and 01: shapes and files are still arrays

pv.Sphere(), pv.Cylinder(), and the rest build the points and faces arrays for you. show_edges=True draws the faces. A Plotter with shape=(3, 3) is a window grid; subplot(row, col) picks a cell of that grid. That shape tuple is not a mesh array. It only splits the window.



Convenience meshes. Each one has a points array of shape (n_points, 3) and a faces array of triangle indices.

Loading data does not change the idea. examples.download_pine_roots() returns a PolyData: points plus faces. examples.download_frog() and ironProt.vtk are ImageData: dimensions, spacing, origin, and a scalar array on the points or cells. dataset.plot(volume=True) colors the volume by that scalar array. pv.read(path) picks the mesh type from the file, then you have the same arrays.

In the volume plot of bolt_nut, cmap="coolwarm" maps the scalar array onto colors, and opacity="sigmoid_5" maps the same numbers onto how solid each voxel looks. The camera direction in that exercise is also an array of 3 points: where the camera sits, what it looks at, and which way is up. Each of those is a length-3 sequence, the same shape as one row of points.

03_figures: arrays that control the picture

- scalars: the name of a point or cell array used for color.
- cmap: which colors that array maps onto. The scalar bar is the legend for that mapping.
- clim or scalar range: a length-2 array [min, max]. Values outside it clamp to the end colors. Trame notebook d_trame_scalar_range is this pair of numbers.
- opacity: one number, or a transfer function of the scalar array.
- camera_position: three length-3 arrays, [position, focal_point, view_up].
- Labels (bonus/e_labels) read a points array or a scalar array and draw text at those coordinates.
- A GIF (d_gif) is a list of frames. Each frame is the same mesh with the camera moved, often by an angle array from np.linspace.

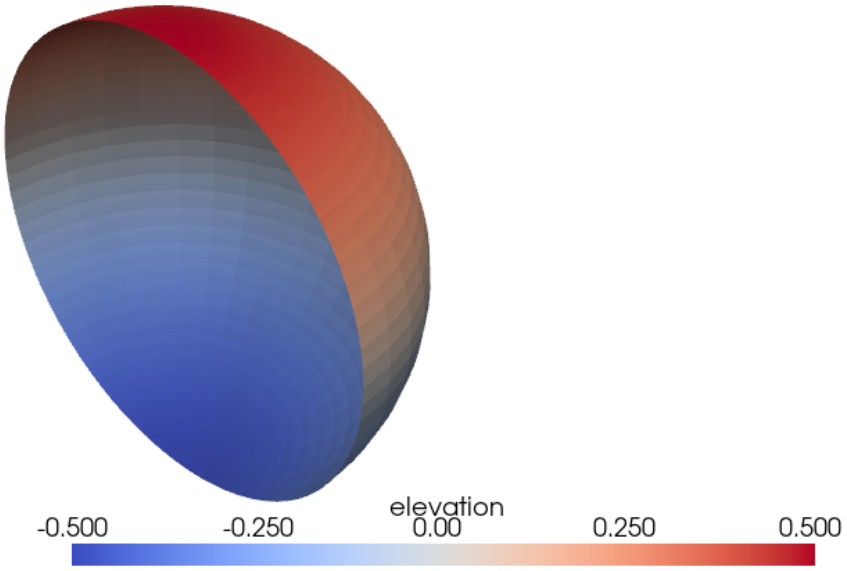
Eye Dome Lighting (exercise c_edl) does not add a data array. It darkens pixels based on depth so a point cloud, which has points but no faces, still shows creases.

04_filters: filters read arrays and return new ones

A filter takes a mesh and returns a new mesh. The new mesh has its own points, faces, and data arrays. Chaining them is the pipeline: output arrays of one step are the input arrays of the next.

Clip

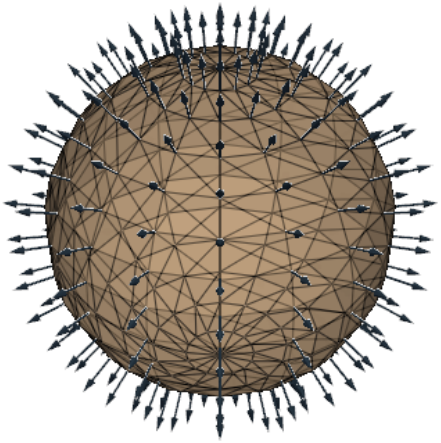
clip(normal="x") keeps points on one side of a plane. normal can be the string "x" or a length-3 array such as (1, 0, 0). The returned mesh has fewer rows in points, and the faces array is rebuilt for the points that remain.



Half the sphere. The scalar array is copied onto the points that survive the cut.

Normals

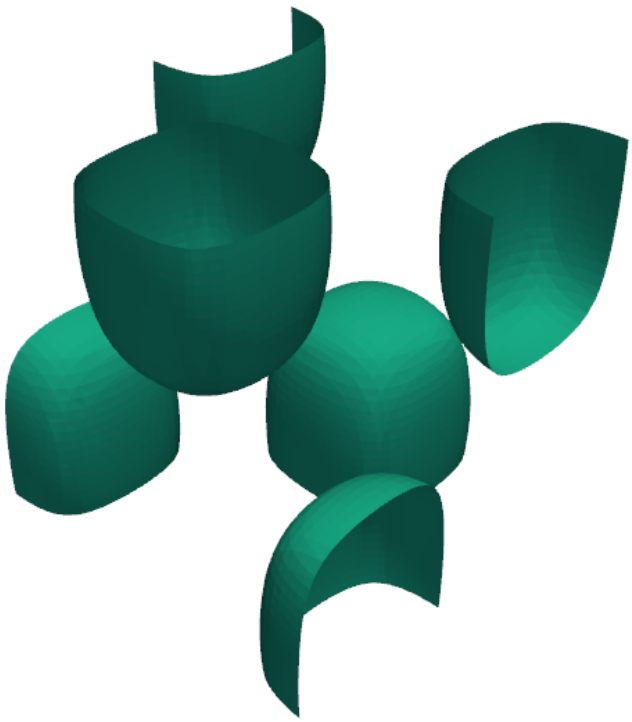
compute_normals writes a new point array of shape (n_points, 3). Each row is a direction sticking out of the surface, usually length 1. Shading uses that array. glyph(orient="Normals") copies an arrow onto each point and rotates it by that row.



The normals array drawn as arrows. One (3,) vector per point.

Contour

contour(isosurfaces=[0.2]) reads a scalar array and builds a new PolyData. The new points lie where the scalar crosses 0.2. Iso-lines are the 2D version of the same cut. The list you pass is the values to extract, not point indices.

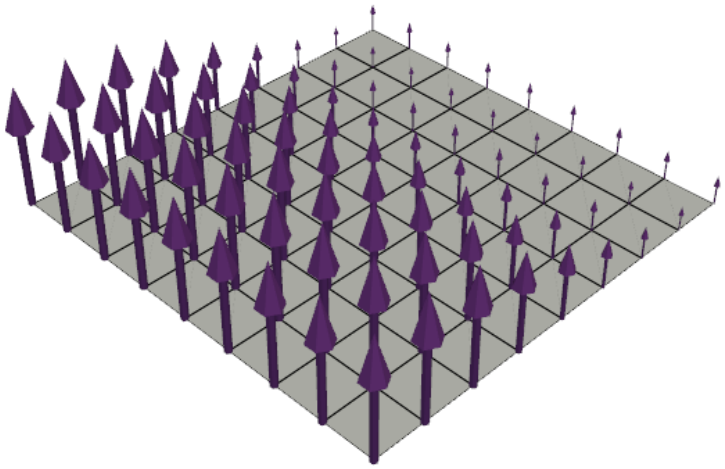


A new points array and faces array, created where a 3D scalar field equals 0.2.

Glyphs

A vector array has shape (n_points, 3): one direction per point. glyph(orient="vectors", scale="vectors") places an arrow at each point. orient chooses the (n, 3) array for direction. scale chooses an array for length. If scale is the same vector array, the length is the size of each vector.

```
plane["vectors"] = np.column_stack([vx, vy, vz])  # shape (n_points, 3)
glyphs = plane.glyph(orient="vectors", scale="vectors", factor=1.2)
```



Arrows from a (n_points, 3) vector array. factor multiplies every length.

The Perlin-noise bonus fills a StructuredGrid by sampling a function onto its points. A StructuredGrid still has an explicit points array of shape (n, 3), unlike ImageData, so the points can be warped. The scalar array is whatever the function returned, one value per point.

06_vtk: the same arrays, in VTK's objects

VTK stores the points array inside a vtkPoints object and the data arrays as VTK arrays. PyVista wraps those. You can convert back and forth.

```
# from b_create_vtk: a uniform image of integers
values = np.arange(np.prod(image.GetDimensions()))
# numpy array -> VTK array, then attach it

# a warped surface: three coordinate arrays joined into (n, 3)
x = np.arange(-10, 10, 0.25)
y = np.arange(-10, 10, 0.25)
x, y = np.meshgrid(x, y)
z = np.sin(np.sqrt(x**2 + y**2))
values = np.c_[x.ravel(), y.ravel(), z.ravel()]   # points, shape (n, 3)
```

vtkImageData matches PyVista ImageData: dimensions, spacing, origin, plus a scalar array. vtkStructuredGrid takes the explicit (n, 3) points, so z = sin(r) can bend the sheet. A VTK algorithm is a filter: you set its input, it writes a new output dataset with new arrays. PyVista's mesh.clip and mesh.contour call those algorithms for you.

d_wasm runs that same pipeline in the browser. e_vtk_next uses the newer VTK API, but the data is still arrays: grid.points.data is the points, and set_array attaches a field such as the magnetic field B, which is a vector array.

08_widgets: a callback edits an array in place

A widget gives you a point (a length-3 array) or a number whenever you drag it. The callback writes that into an existing array. The mesh updates because its points or scalars changed, not because you built a new window.

- Sphere widget: callback(point) receives one row. The triangle example does surf.points[0] = point, which replaces row 0 of the (n, 3) points array.
- Several sphere widgets pass an index too: surf.points[i] = point.
- Line widget: the two ends are length-3 arrays, often called pointa and pointb. The streamline call samples between them.
- Slider: the callback gets one float. You use it to change a threshold, a scale, or one entry of a scalar array.
- Box and plane widgets: the box or the plane is defined by a few length-3 arrays (origin, bounds, normal). The callback usually clips with those.
- Spline widget: initial_points is an (n_handles, 3) array. Dragging a handle rewrites one row, and the curve is rebuilt through the rows.

```
# from g_sphere-widget, example A
surf.points = np.array([[0, 0, 0], [1, 0, 0], [0.5, 1, 0]])  # shape (3, 3)
surf.faces = np.array([3, 0, 1, 2])
def callback(point):
    surf.points[0] = point    # overwrite row 0; indices in faces stay valid
```

09_trame: the web page picks which array to show

Trame does not invent a new mesh. The page holds a PyVista mesh and a few controls that change which array is active.

- a_trame_simple / a_getting_started: show one mesh.
- b_trame_actor_color: the actor is the drawn mesh. The control sets its color. The color list is a Python dict, not a mesh array.
- c_trame_scalars: mesh.point_data.keys() is the list of point-array names. The dropdown sets actor.mapper.array_name to one of those names.
- d_trame_scalar_range: two numbers, the min and max used to map that array onto the colormap.
- e_trame_algorithm: a button or slider runs a filter and replaces the mesh, which replaces points, faces, and data arrays.
- f_trame_open_file: pv.read loads a file into those same arrays.

Shapes to check before you plot

When a picture looks wrong, check the shape first.

- points: (n_points, 3). A single point you pass to a callback is (3,).
- faces: 1D. Each cell is [count, i0, i1, ...]. The i's are row numbers in points.
- point scalar array: (n_points,). cell scalar array: (n_cells,).
- vector array (normals, glyphs, velocity): (n_points, 3).
- ImageData dimensions: (nx, ny, nz). Point data flattens to nx*ny*nz values. Cell data needs dimensions = array_shape + 1.
- Flatten 3D arrays with order="F" before assigning to point_data or cell_data.
- camera_position: three arrays of length 3.

Where each folder uses this

- 00_intro: PyVista builds points and faces for Cylinder, Arrow, Sphere, Plane, Line, Box.
- 00_jupyter: same meshes, drawn with the trame, client, or server backend.
- 01_basic: read a file or examples.download_... and plot the arrays that come back.
- 02_mesh: you build the arrays yourself. Exercises cover a point cloud, a uniform grid, a triangulated surface, and reading a file.
- 03_figures: color, light, subplots, labels, orbit, GIF. These choose how an array is drawn.
- 04_filters: clip, normals, contour, glyphs, Perlin noise. New arrays out.
- 06_vtk: the same arrays inside VTK objects, including the newer API.
- 08_widgets: callbacks write into points or scalar arrays while you drag.
- 09_trame: a web control selects an array name or a scalar range.

Exercises are the practice notebooks. solutions folders are the filled-in arrays. Bonus notebooks are optional. This copy has no lesson notebooks in 05_action and no 07 folder.

Lessons come from the official PyVista tutorial notebooks.